

DOCUMENTACIÓN DEL DESARROLLO

1. ¿Para qué sirve el programa?

El programa es un **Escáner de Red Local (LAN)** realizado en Java. Su función principal es analizar un rango de direcciones IP dentro de una misma subred (por ejemplo, de la 192.168.0.1 a la 192.168.0.15) y determinar qué dispositivos están conectados a esa red y activos

De cada dispositivo conectado se muestra la siguiente informacion

- Dirección IPv4.
- Nombre en la red (o "No resuelto" si no lo identifica).
- Estado ("Alcanzable").
- Tiempo de respuesta o latencia en milisegundos.

Se puede guardar el resultado obtenido en un archivo de texto (.txt) que se guarda directamente en la carpeta Documentos del equipo

2. Cómo está armado el sistema

El proyecto utiliza el patrón de diseño **Modelo-Vista-Controlador (MVC)** estructurado en las siguientes carpetas:

- EscanerDeRed/
 - src/
 - controlador/
 - LogicaEscaner.java
 - modelo/
 - DispositivoEncontrado.java
 - utilidades/
 - ValidarIpValida.java
 - vista/
 - VentanaVisualEscaner.java
 - App.java

3. Métodos utilizados y funcionalidad

- **LogicaEscaner.probarIp(ip, timeout):**
 - *Uso:* Realiza el ping mediante `InetAddress.getByName(ip)` e `inet.isReachable(timeout)`. Mide el tiempo de inicio y fin con `System.currentTimeMillis()`. Si la latencia es menor o igual al timeout ingresado, retorna un los datos en `DispositivoEncontrado`.
- **ValidarIpValida.verificarFormato(textoIp):**
 - *Uso:* Comprueba mediante una expresión regular (`REGEX_IP`) que las cadenas ingresadas cumplan con el formato de una dirección IPv4 válida (192.128.0.1 por ejemplo).
- **VentanaVisualEscaner.empezarEscaneo():**
 - *Uso:* Lee las IPs y el timeout de la interfaz, valida que la IP inicial no sea mayor a la final y que pertenezcan a la misma subred. Recorre el rango mediante un bucle for llamando a `LogicaEscaner.probarIp()` y agrega cada resultado directamente a la tabla.
- **VentanaVisualEscaner.guardarEnDocumentos():**
 - *Uso:* Detecta la ruta del usuario con `System.getProperty("user.home")`, localiza la carpeta de documentos y genera el archivo `reporte_escaneo.txt` formateado en columnas.

4. Selección de tecnologías

- **Java SE:** Lenguaje base por su portabilidad y soporte nativo de APIs de red
- **Java Swing:** Framework gráfico nativo para construir la interfaz de usuario sin requerir librerías externas
- **java.net.InetAddress:** API estándar utilizada para verificar la alcanzabilidad de hosts y resolver nombres de dominio locales

5. Problemas encontrados y soluciones

1. **Inexactitud del Timeout en Windows:** `isReachable()` puede demorar más de lo especificado por cuestiones de permisos o descarte del sistema operativo.
 - *Solución:* Se implementó una verificación explícita del tiempo transcurrido (`fin - inicio`), descartando cualquier respuesta que supere el límite ingresado.
2. **Validación de datos:** Evitar que el usuario ingrese rangos de red diferentes o textos inválidos.
 - *Solución:* Se añadieron controles de validación y comparación del prefijo de subred antes de iniciar el bucle de escaneo.

6. Mejoras futuras

- Exportar reportes en formatos estructurados adicionales como CSV o JSON.